

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №0
по курсу «Алгоритмы и структуры данных»
Тема: Введение
Вариант 1

Выполнил:
Степанова Е. А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

<u>Содержание отчета</u>	2
<u>Задачи по варианту</u>	3
<u>Задача №1. Ввод-вывод [N баллов]</u>	
<u>Задача №2. Число Фибоначчи[N баллов]</u>	13
<u>Задача №3. Еще про число Фибоначчи[N баллов]</u>	16
<u>Задача №4. Тестирование ваших алгоритмов [N баллов]</u>	18
Вывод	22

Задачи по варианту

Задача №1. Ввод-вывод [N баллов]

ПОДЗАДАЧА 1

```
a, b = map(int, input().split())
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    print(a + b)
```

```
a, b = map(int, input().split())
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    print(a + b)
```

Необходимо найти сумму чисел введенных пользователем.

В начале считываем переменные a и b для этого воспользуемся функцией `map`, т.к. 2 числа подаются в одной строке через разделитель, в нашем случае через пробел.

По условию $-10^9 \leq a, b \leq 10^9$, поэтому проверяю это условие, а дальше вывожу сумму чисел, если условие выполнилось.

```
-1000000000 -1000000000
-2000000000
```

```
12 25
37
```

```
130 61
191
```

```
1000000000 1000000000
2000000000
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.2000000424450263e-06 секунд	22.3 MiB
Пример из задачи	1.3000008038943633e-06 секунд	22.3 MiB
Пример из задачи	1.0999992809956893e-06 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	1.3000008038943633e-06 секунд	22.4 MiB

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(максимальных, минимальных и из данного примера) по времени программа работает примерно одинаково, аналогично и с затраченной памятью, подсчитывает как минимальные так и максимальные значения довольно быстро - нет проблем со сложность для данного диапазона входных данных.

ПОДЗАДАЧА 2

```
a, b = map(int, input().split())
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    print(a + b ** 2)
```

```
a, b = map(int, input().split())
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    print(a + b ** 2)
```

В начале считываем переменные a и b для этого воспользуемся функцией `map`, т.к. 2 числа подаются в одной строке через разделитель, в нашем случае через пробел.

По условию $-10^9 \leq a, b \leq 10^9$, поэтому проверяю это условие, а дальше вывожу необходимое выражение $(a+b^2)$, если условие выполнилось.

```
-1000000000 -1000000000
999999999000000000
```

```
1000000000 1000000000
10000000001000000000
```

```
1 1
2
```

```
10 10
110
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.0000003385357559e-06 секунд	22.3 MiB
Пример	1.5000005078036338e-06 секунд	22.3 MiB
Пример	1.6000012692529708e-06 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	1.7000020307023078e-06 секунд	22.3 MiB

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(собственных примеров, максимальных и минимальных) время затрачивается разное, при более больших значениях программа работает дольше, но затрачивается одинаковое количество памяти, подсчитывает как минимальные так и максимальные значения довольно быстро - нет проблем со сложность для данного диапазона входных данных.

ПОДЗАДАЧА 3

```
a, b = open('input.txt').read().split()
a, b = int(a), int(b)
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    open('output.txt', 'w').write(str(a + b))
```

```
a, b = open('input.txt').read().split()
a, b = int(a), int(b)
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    open('output.txt', 'w').write(str(a + b))
```

Открываем файл, тип не указала, по умолчанию - чтение. далее с помощью read() считываю данные из файла, разделяю строку по пробелу с помощью split(), присваиваю полученные значения переменным a и b. Проверяю необходимое условие. Если оно выполняется, то открываю файл, в который необходимо записать ответ ('output.txt') с типом 'w', чтобы воспользоваться методом write и записать результат суммы переменных a и b в файл('output.txt').

task.3.py 1_task.4.py 2_task.py 3_task.py × output.txt ×

1	37
---	----

✓

АиСД - input.txt

input.txt ×

1	12 25
---	-------

_task.3.py 1_task.4.py 2_task.py 3_task.py × output.txt ×

1	191
---	-----

✓

АиСД - input.txt

input.txt ×

1	130 61
---	--------

ask.4.py 2_task.py 1_task.3.py × 3_task.py output.txt ×

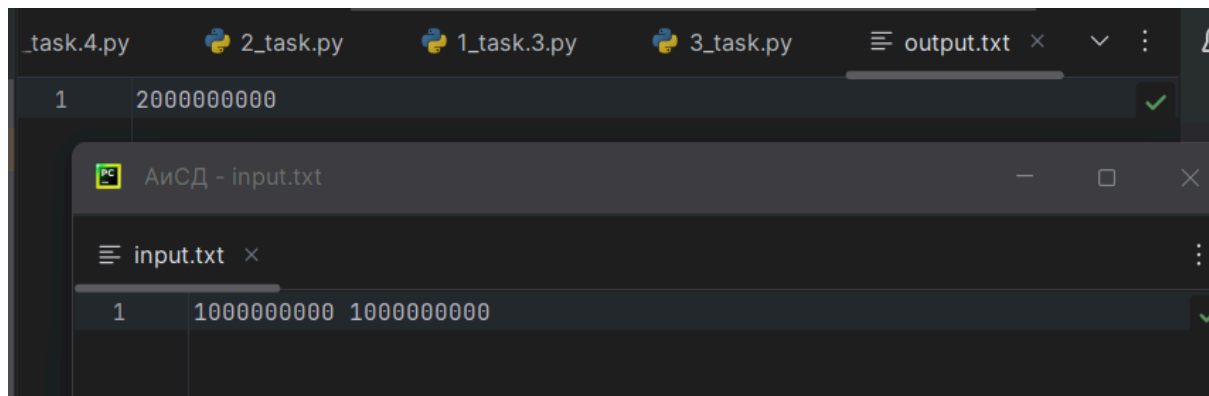
1	-20000000000
---	--------------

✓

АиСД - input.txt

input.txt ×

1	-10000000000 -10000000000
---	---------------------------



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.0999974620062858e-06 секунд	22.3 MiB
Пример из задачи	1.399997463542968e-06 секунд	22.3 MiB
Пример из задачи	1.5999976312741637e-06 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	1.5999976312741637e-06 секунд	22.3 MiB

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(максимальных, минимальных и из примеров) время затрачивается разное, при более больших значениях программа работает дольше, но затрачивается примерно одинаковое количество памяти, подсчитывает как минимальные так и максимальные значения довольно быстро - нет проблем со сложностью для данного диапазона входных данных.

ПОДЗАДАЧА 4

```
a, b = open('input.txt').read().split()
a, b = int(a), int(b)
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    open('output.txt', 'w').write(str(a + b ** 2))
```

```
a, b = open('input.txt').read().split()
a, b = int(a), int(b)
if -10 ** 9 <= a <= 10 ** 9 and -10 ** 9 <= b <= 10 ** 9:
    open('output.txt', 'w').write(str(a + b ** 2))
```

Открываем файл, тип не указала, по умолчанию - чтение. далее с помощью read() считываю данные из файла, разделяю строку по пробелу с помощью split(), присваиваю полученные значения переменным a и b. Проверяю необходимое условие. Если оно выполняется, то открываю файл, в который необходимо записать ответ ('output.txt') с типом 'w', чтобы воспользоваться методом write и записать результат $(a+b^2)$ в файл('output.txt').

```
_task.4.py 2_task.py 1_task.3.py 3_task.py output.txt x
1 9999999990000000000
PC АиСД - input.txt
input.txt x
1 -10000000000 -10000000000
```

```
_task.4.py 2_task.py 1_task.3.py 3_task.py output.txt x v
1 10000000001000000000
PC АиСД - input.txt
input.txt x
1 10000000000 10000000000
```

```
_task.4.py 2_task.py 1_task.3.py 3_task.py output.txt x v
1 2 ✓
PC АиСД - input.txt
input.txt x
1 1 1
```

```
_task.4.py 2_task.py 1_task.3.py 3_task.py output.txt x
1 110
PC АиСД - input.txt
input.txt x
1 10 10
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.5000005078036338e-06 секунд	22.2 MiB
Пример	1.3999961083754897e-06 секунд	22.3 MiB
Пример	1.7999991541728377e-06 секунд	22.3 MiB
Верхняя граница диапазона значений входных данных из текста задачи	1.500004145782441e-06 секунд	22.4 MiB

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(максимальных, минимальных и из собственных примеров) время затрачивается разное, какой-то закономерности не увидела, но с увеличением входных значений увеличиваются затраты памяти, подсчитывает как минимальные так и максимальные значения довольно быстро - нет проблем со сложностью для данного диапазона входных данных.

Задача №2. Числа Фибоначчи [N баллов]

```
def fib(n):  
    f_prev = 0  
    f_new = 1  
    for k in range(n):  
        f_prev, f_new = f_new, f_new + f_prev  
    return f_prev  
  
f1 = open('input.txt', 'r')  
n = int(f1.readline())  
f1.close()  
if 0 <= n <= 45:  
    f2 = open('output.txt', 'w')  
    f2.write(str(fib(n)))  
    f2.close()
```

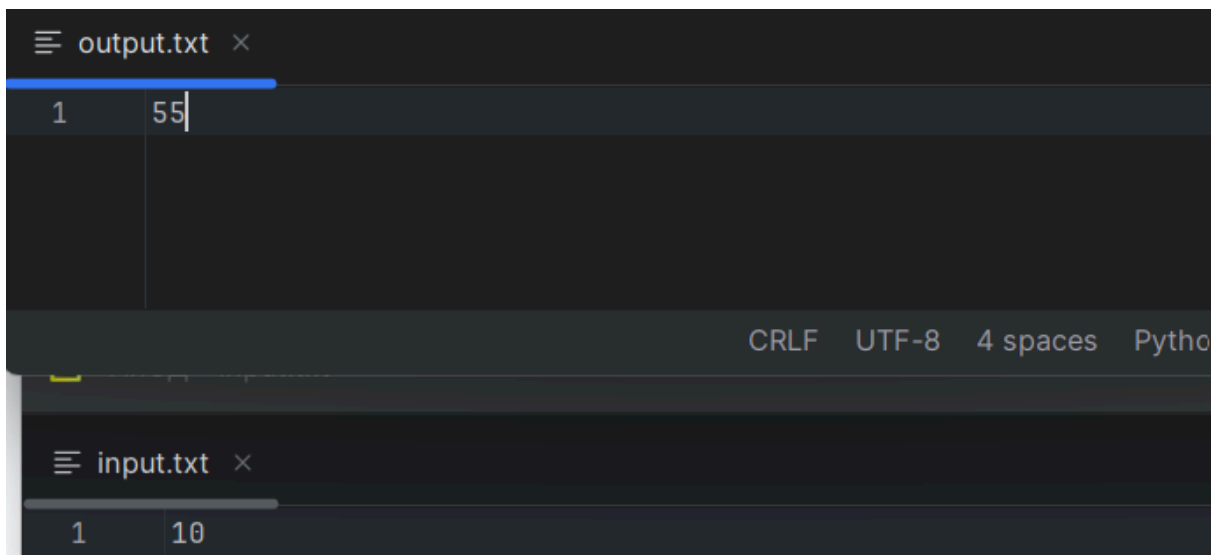
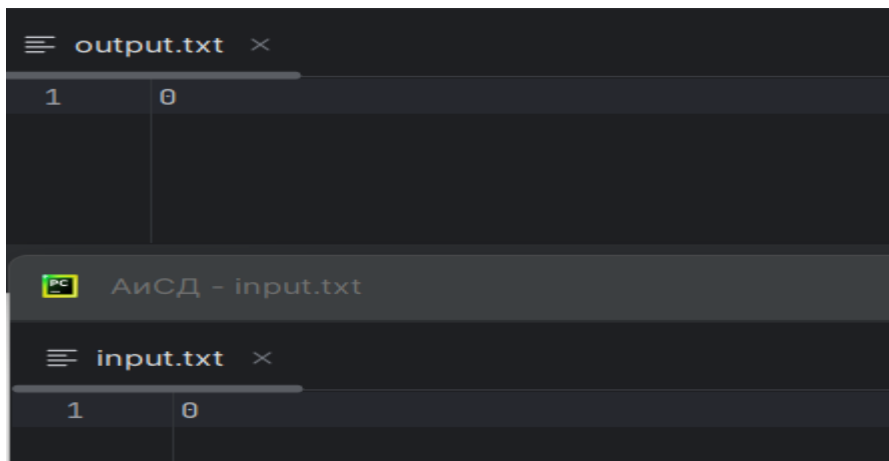
```
def fib(n):  
    f_prev = 0  
    f_new = 1  
    for k in range(n):  
        f_prev, f_new = f_new, f_new + f_prev  
    return f_prev
```

```
f1 = open('input.txt', 'r')  
n = int(f1.readline())  
f1.close()  
if 0 <= n <= 45:  
    f2 = open('output.txt', 'w')
```

```
f2.write(str(fib(n)))  
f2.close()
```

Первое: пишу функцию для вычисления чисел фибоначчи, известно, что первые два элемента это 0 и 1, далее проходим по циклу с n заданными итерациями, заменяя f_new на сумму предыдущего числа и того, который ранее хранился в f_new, а в переменную, отвечающую за предыдущее число f_prev записываю старое значение переменной f_new, возвращаю f_prev.

Второе: открываю файл, считываю значение и присваиваю его переменной n, проверяю условие, если верно, то открываю файл для записи ответа, записываю методом write результат функции при значении n, присвоив ему строковый тип данных.



```

АиСД - output.txt
output.txt x
1 1134903170

input.txt x
1 45
  
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.0999938240274787e-06 секунд	22.3 MiB
Пример из задачи	1.4999968698248267e-06 секунд	22.4 MiB
Пример из задачи	-	-
Верхняя граница диапазона значений входных данных из текста задачи	1.5999976312741637e-06 секунд	22.2 MiB

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(максимальных, минимальных и из примеров) время затрачивается разное, при увеличении входных данных увеличивается время работы и затраты на память, подсчитывает как минимальные так и

максимальные значения довольно быстро - нет проблем со сложностью для данного диапазона входных данных.

Задача №3. Еще про число Фибоначчи [N баллов]

```
def fib(n):
    f_prev = 0
    f_new = 1
    for k in range(n):
        f_prev, f_new = f_new, f_new + f_prev
    return f_prev

def last_digit():
    ans = fib(n) % 10
    return ans

f1 = open('input.txt', 'r')
n = int(f1.readline())
f1.close()
if 0 <= n <= 10 ** 7:
    f2 = open('output.txt', 'w')
    f2.write(str(last_digit()))
    f2.close()
```

```
def fib(n):
    f_prev = 0
    f_new = 1
    for k in range(n):
        f_prev, f_new = f_new, f_new + f_prev
    return f_prev
```

```
def last_digit():
    ans = fib(n) % 10
    return ans
```

```
f1 = open('input.txt', 'r')
n = int(f1.readline())
```

```

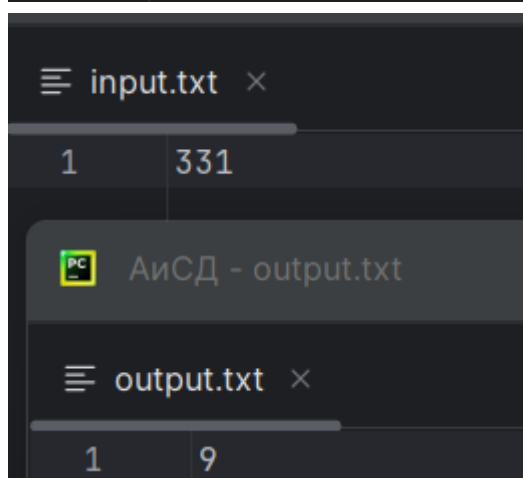
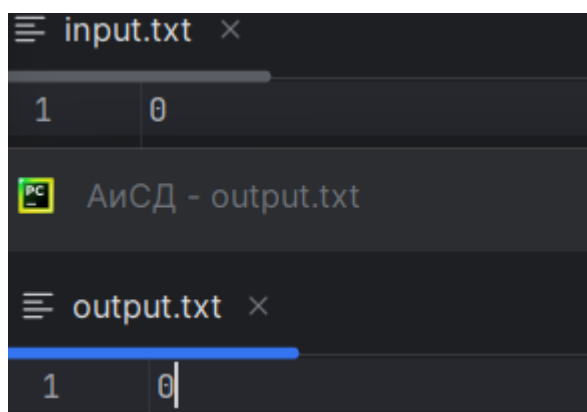
f1.close()
if 0 <= n <= 10**7:
    f2 = open('output.txt', 'w')
    f2.write(str(last_digit()))
    f2.close()

```

Первое: пишу функцию для вычисления чисел фибоначчи, известно, что первые два элемента это 0 и 1, далее проходим по циклу с n заданными итерациями, заменяя f_new на сумму предыдущего числа и того, который ранее хранился в f_new, а в переменную, отвечающую за предыдущее число f_prev записываю старое значение переменной f_new, возвращаю f_prev.

Второе: сделала отдельную функцию last_digit() для вычисления последней цифры большого числа фибоначчи, для этого достаточно взять остаток от деления результата fib() на 10.

Далее, открываю файл, считываю значение и присваиваю его переменной n, проверяю условие, если верно, то открываю файл для записи ответа, записываю методом write результат функции при значении n, присвоив ему строковый тип данных.



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.5999976312741637e-06 секунд	22.3 MiB
Пример из задачи	1.6999983927235007e-06 секунд	22.2 MiB
Пример из задачи	1.7999991541728377e-06 секунд	23.0 MiB
Верхняя граница диапазона значений входных данных из текста задачи		

Вывод по задаче: при анализе таблицы видно, что при подстановке разных значений(максимальных, минимальных и из примеров) время затрачивается разное, при увеличении входных данных увеличивается время работы и затраты на память. Проводила тест для верхней границы диапазона значений данных из текста задачи (10^7), из-за того, что значение слишком большое ответ он считает слишком долго, не вывел ничего, не помогла использование `lru_cache`(из библиотеки `functools`) и `setrecursionlimit`(из библиотеки `sys`), из этого следует, что мой алгоритм не подходит для больших входных данных и имеет проблемы со сложность.

Задача №4. Тестирование ваших алгоритмов[N баллов]

Для задачи 3

```
import time

from memory_profiler import profile
```

```

t_start = time.perf_counter()

@profile
def main():
    def fib(n):
        f_prev = 0
        f_new = 1
        for k in range(n):
            f_prev, f_new = f_new, f_new + f_prev
        return f_prev

    def last_digit():
        ans = fib(n) % 10
        return ans

    f1 = open('input.txt', 'r')
    n = int(f1.readline())
    f1.close()
    if 0 <= n <= 10 ** 7:
        f2 = open('output.txt', 'w')
        f2.write(str(last_digit()))
        f2.close()

if __name__ == '__main__':
    main()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
      (time.perf_counter() - t_start))

```

Для подсчета времени использую библиотеку `time`. Ввожу переменную `t_start`, в которой применяю функцию `time.perf_counter()`, которая измеряет временной интервал с точностью до доли секунды, что делает его пригодным для измерения производительности и сравнительного анализа. После основного кода я останавливаю таймер и вывожу значение. Также в этом коде можно заметить библиотеку `memory_profiler`, я ее использую, чтобы измерить затраты памяти.

Line #	Mem usage	Increment	Occurrences	Line Contents
8	22.2 MiB	22.2 MiB	1	@profile
9				def main():
10	22.2 MiB	0.0 MiB	2	def fib(n):
11	22.2 MiB	0.0 MiB	1	f_prev = 0
12	22.2 MiB	0.0 MiB	1	f_new = 1
13	22.2 MiB	0.0 MiB	21	for k in range(n):
14	22.2 MiB	0.0 MiB	20	f_prev, f_new = f_new
15	22.2 MiB	0.0 MiB	1	return f_prev
16				
17	22.2 MiB	0.0 MiB	2	def last_digit():
18	22.2 MiB	0.0 MiB	1	ans = fib(n) % 10
19	22.2 MiB	0.0 MiB	1	return ans
20				
21	22.2 MiB	0.0 MiB	1	f1 = open('input.txt', 'r')
22	22.2 MiB	0.0 MiB	1	n = int(f1.readline())
23	22.2 MiB	0.0 MiB	1	f1.close()
24	22.2 MiB	0.0 MiB	1	if 0 <= n <= 10 ** 7:
25	22.2 MiB	0.0 MiB	1	f2 = open('output.txt', 'w')
26	22.2 MiB	0.0 MiB	1	f2.write(str(last_digit(n)))
27	22.2 MiB	0.0 MiB	1	f2.close()

Время работы: 2.1000014385208488e-06 секунд

Вот результат вывода, если входное значение равно 20.

Line #	Mem usage	Increment	Occurrences	Line Contents
=====				
8	22.3 MiB	22.3 MiB	1	@profile
9				def main():
10	22.3 MiB	0.0 MiB	2	def fib(n):
11	22.3 MiB	0.0 MiB	1	f_prev = 0
12	22.3 MiB	0.0 MiB	1	f_new = 1
13	22.3 MiB	0.0 MiB	2	for k in range(n):
14	22.3 MiB	0.0 MiB	1	f_prev, f_new = f_new
15	22.3 MiB	0.0 MiB	1	return f_prev
16				
17	22.3 MiB	0.0 MiB	2	def last_digit():
18	22.3 MiB	0.0 MiB	1	ans = fib(n) % 10
19	22.3 MiB	0.0 MiB	1	return ans
20				
21	22.3 MiB	0.0 MiB	1	f1 = open('input.txt', 'r')
22	22.3 MiB	0.0 MiB	1	n = int(f1.readline())
23	22.3 MiB	0.0 MiB	1	f1.close()
24	22.3 MiB	0.0 MiB	1	if 0 <= n <= 10 ** 7:
25	22.3 MiB	0.0 MiB	1	f2 = open('output.txt',
26	22.3 MiB	0.0 MiB	1	f2.write(str(last_digit()
27	22.3 MiB	0.0 MiB	1	f2.close()
Время работы: 1.5999976312741637e-06 секунд				

Вот результат вывода, если входное значение равно 1.

Для задачи 2

```
import time

t_start = time.perf_counter()

from memory_profiler import profile

@profile
def main():

    def fib(n):
        f_prev = 0
        f_new = 1
```

```

        for k in range(n):
            f_prev, f_new = f_new, f_new + f_prev
        return f_prev

f1 = open('input.txt', 'r')
n = int(f1.readline())
f1.close()
if 0 <= n <= 45:
    f2 = open('output.txt', 'w')
    f2.write(str(fib(n)))
    f2.close()
if __name__ == '__main__':
    main()
t_start = time.perf_counter()
print("Время выполнения: %s секунд" %
      (time.perf_counter() - t_start))

```

Аналогично задаче 3, для подсчета времени использую библиотеку `time`. Ввожу переменную `t_start`, в которой применяю функцию `time.perf_counter()`, которая измеряет временной интервал с точностью до доли секунды, что делает его пригодным для измерения производительности и сравнительного анализа. После основного кода я останавливаю таймер и вывожу значение.

Также в этом коде можно заметить библиотеку `memory_profiler`, я ее использую, чтобы измерить затраты памяти.

Вывод по задаче: необходимо смотреть на время выполнения программы и затраты памяти, сделать это можно с помощью библиотеки `time` и `memory_profiler`.

Вывод

при написании программы необходимо проверять не только корректность выполнения на каких-то небольших примерах, важно рассмотреть как можно больше чисел из диапазона: максимальные, минимальные и некоторые числа, которые лежат в промежутке между мин и макс, благодаря этому можно увидеть эффективность своего алгоритма, оценить сколько времени и памяти он займет при тех или иных значениях, будет ли алгоритм эффективен при больших значениях или при маленьких и тд.

Для себя я узнала, что чтобы корректно оценить время и затраты на память, можно использовать библиотеку `time` и `memory_prifiler`.

Это важно, т.к. если программа будет считать очень долго, это может привести к ошибкам или вообще результат не будет выведен(например, как в задаче 3), поэтому при написании кода важно проанализировать диапазон значений(входных данных) и операции, которые будут производиться(так например, если диапазон будет слишком большой и будут выполняться сложные операции, то программу точно нужно будет оптимизировать)

